main.py

Visualize

Run ▶

```python
from collections import deque

CAP_A, CAP_B, TARGET = 4, 3, 2

def successors(state):
    a, b = state
    pour_ab = min(a, CAP_B - b)
    pour_ba = min(b, CAP_A - a)
    return {
        (CAP_A, b): "Fill 4L jug",
        (a, CAP_B): "Fill 3L jug",
        (0, b): "Empty 4L jug",
        (a, 0): "Empty 3L jug",
        (a - pour_ab, b + pour_ab): "Pour 4L -> 3L",
        (a + pour_ba, b - pour_ba): "Pour 3L -> 4L",
    }

def solve():
    start = (0, 0)
    queue = deque([start])
    parent = {start: (None, "Start")}
    while queue:
        cur = queue.popleft()
        if cur[0] == TARGET:
            path = []
            while cur is not None:
```

main.py

Visualize

Run ▶

```
        (0, b): "Empty 4L jug",
        (a, 0): "Empty 3L jug",
        (a - pour_ab, b + pour_ab): "Pour 4L -> 3L",
        (a + pour_ba, b - pour_ba): "Pour 3L -> 4L",
    }

def solve():
    start = (0, 0)
    queue = deque([start])
    parent = {start: (None, "Start")}
    while queue:
        cur = queue.popleft()
        if cur[0] == TARGET:
            path = []
            while cur is not None:
                path.append((cur, parent[cur][1]))
                cur = parent[cur][0]
            return path[::-1]
        for nxt, action in successors(cur).items():
            if nxt not in parent:
                parent[nxt] = (cur, action)
                queue.append(nxt)

for state, action in solve():
    print(state, "<-", action)
```

Output

```
(0, 0) <- Start
(4, 0) <- Fill 4L jug
(1, 3) <- Pour 4L -> 3L
(1, 0) <- Empty 3L jug
(0, 1) <- Pour 4L -> 3L
(4, 1) <- Fill 4L jug
(2, 3) <- Pour 4L -> 3L
```